Vaultwarden Security Audit Report

Part 3: Medium & Low Severity Findings

Audit Date: January 2026 **Application:** Vaultwarden v1.0.0 **Auditor:** Security Assessment Team **Repository:** https://github.com/dani-garcia/vaultwarden

Executive Summary

This report details the **3 Medium severity** and **5 Low severity** findings identified during the comprehensive security audit of Vaultwarden. While the application demonstrates strong security fundamentals, these findings represent areas where security controls could be enhanced to meet industry best practices and compliance requirements (particularly SOC 2).

Findings Overview

Medium Severity (3): 1. Plain-text ADMIN_TOKEN acceptance (legacy support) 2. Complex security stamp exception logic 3. Password hints stored in plaintext

Low Severity (5): 1. Limited admin action audit logging 2. Missing WebSocket rate limiting 3. No MFA option for admin panel 4. Event log export functionality gaps 5. Input validation edge cases

Medium Severity Findings

M-1: Plain-text ADMIN_TOKEN Acceptance (Legacy Support)

Severity: Medium **CWE:** CWE-256 (Plaintext Storage of a Password) **SOC 2 Relevance:** Yes (CC6.1 - Logical and Physical Access Controls)

Description

The admin panel authentication accepts both Argon2id hashed tokens and plain-text tokens for backward compatibility. While the system recommends Argon2id hashing, it does not enforce it,

allowing administrators to configure plain-text admin tokens that are vulnerable to exposure through configuration files, logs, or memory dumps.

Affected Code

File: `src/api/admin.rs:229-247`

```
fn _validate_token(token: &str) -> bool {
    match CONFIG.admin_token().as_ref() {
        None => false,
        Some(t) if t.starts_with("$argon2") => {
            use argon2::password_hash::PasswordVerifier;
            match argon2::password_hash::PasswordHash::new(t) {
                Ok(h) => {
                    // NOTE: hash params from `ADMIN_TOKEN` are used instead of what is
                    argon2::Argon2::default().verify_password(token.trim().as_ref(), &h
                }
                Err(e) => {
                    error!("The configured Argon2 PHC in `ADMIN_TOKEN` is invalid: {e}"
                    false
                }
            }
        }
        Some(t) => crate::crypto::ct_eq(t.trim(), token.trim()),  // ⚠️ Plain-text comp
    }
}
```

Configuration: `.env.template` or `config.json`

```
# Plain-text token (INSECURE - legacy support)
ADMIN_TOKEN=my_secret_admin_password

# Argon2id hashed token (RECOMMENDED)
ADMIN_TOKEN=$argon2id$v=19$m=65540,t=3,p=4$...
```

Risk Analysis

Attack Vectors: 1. **Configuration File Exposure:** If `.env` or `config.json` files are exposed through misconfiguration (e.g., web server serving dotfiles, backup files in web root, git commits), the plain-text admin token is immediately compromised. 2. **Log File Leakage:**

Support diagnostics or debugging outputs could inadvertently log the plain-text token. 3. **Memory Dumps:** Plain-text tokens in memory are vulnerable to memory scraping attacks or core dumps. 4. **Insider Threats:** System administrators with file access can read the plain-text token directly. 5. **Backup Exposure:** Configuration backups stored insecurely expose the token.

Impact: - Complete administrative access to Vaultwarden instance - Ability to view all user accounts, organizations, and encrypted data metadata - Capability to delete users, disable 2FA, modify system configuration - Potential data exfiltration of user emails, encrypted vault metadata, organization structure - Compromise of audit trail integrity

Likelihood: Medium (common misconfiguration scenario)

Exploitation Scenario

```
# Attacker gains read access to server filesystem via:
# - Path traversal vulnerability in another service
# - Exposed .git directory (git-dumper tool)
# - Compromised backup system
# - Insider access

# 1. Read configuration file
$ cat /vaultwarden/.env | grep ADMIN_TOKEN
ADMIN_TOKEN=MySecretPassword123

# 2. Access admin panel
$ curl -X POST https://vault.example.com/admin \
  -d "token=MySecretPassword123" \
  -c cookies.txt

# 3. Extract all user information
$ curl https://vault.example.com/admin/users \
  -b cookies.txt \
  -H "Accept: application/json"
{
  "users": [
    {"email": "user1@example.com", "uuid": "...", ...},
    {"email": "user2@example.com", "uuid": "...", ...}
  ]
}

# 4. Disable 2FA for target accounts
$ curl -X POST https://vault.example.com/admin/users/<uuid>/remove-2fa \
  -b cookies.txt

# 5. Modify system configuration to disable security features
$ curl -X POST https://vault.example.com/admin/config \
  -b cookies.txt \
  -d '{"signups_allowed": true, "invitations_allowed": true}'
```

Remediation

Priority: High

Immediate Actions: 1. **Add startup warning** when plain-text ADMIN_TOKEN is detected:

```
if let Some(token) = CONFIG.admin_token() {
    if !token.starts_with("$argon2") {
        warn!("⚠️  SECURITY WARNING: Plain-text ADMIN_TOKEN detected!");
        warn!("⚠️  Please hash your admin token using Argon2id.");
        warn!("⚠️  Run: vaultwarden hash --preset owasp");
    }
}
```

1. **Documentation update:** Prominently display security warning in wiki and installation guides.

2. **Migration tool:** Provide command-line utility to hash existing plain-text tokens:

```
$ vaultwarden hash --preset owasp
Enter admin token: ****
Argon2id hash: $argon2id$v=19$m=65540,t=3,p=4$...

Add this to your .env file:
ADMIN_TOKEN='$argon2id$v=19$m=65540,t=3,p=4$...'
```

Long-term Actions (Breaking Change for v2.0): 1. **Deprecate plain-text tokens:** Remove `Some(t) => crate::crypto::ct_eq()` fallback. 2. **Enforce Argon2id hashing:** Only accept `$argon2` PHC strings. 3. **Configuration validation:** Fail to start if ADMIN_TOKEN is not properly hashed.

Compensating Controls: - Ensure strict file permissions on `.env` and `config.json` (0600 or 0400) - Use environment variables instead of config files where possible - Implement configuration management with encrypted secrets (e.g., Vault, SOPS, sealed-secrets) - Enable audit logging for admin panel access - Implement IP allowlisting for admin panel if feasible

Testing:

```
#[test]
fn test_plaintext_token_rejected() {
    // Future: ensure plain-text tokens are rejected
    let result = _validate_token("plain_text_password");
    assert!(!result, "Plain-text tokens should be rejected");
}
```

M-2: Complex Security Stamp Exception Logic

Severity: Medium **CWE:** CWE-691 (Insufficient Control Flow Management) **SOC 2 Relevance:** Yes (CC6.1 - Logical and Physical Access Controls)

Description

Vaultwarden implements a "security stamp exception" mechanism to allow users to complete multi-step operations (e.g., account recovery, security key registration) without being immediately logged out when their security stamp changes. This exception logic is complex, involving time-based expiration (2 minutes), route allowlisting, and stamp matching. The complexity introduces potential for logic errors, race conditions, or bypass scenarios.

Affected Code

File: `src/auth.rs:626-654`

```
if user.security_stamp != claims.sstamp {
    if let Some(stamp_exception) =
        user.stamp_exception.as_deref().and_then(|s| serde_json::from_str::<UserStampEx
    {
        let Some(current_route) = request.route().and_then(|r| r.name.as_deref()) else
            err_handler!("Error getting current route for stamp exception")
        };

        // Check if the stamp exception has expired first.
        // Then, check if the current route matches any of the allowed routes.
        // After that check the stamp in exception matches the one in the claims.
        if Utc::now().timestamp() > stamp_exception.expire {
            // If the stamp exception has been expired remove it from the database.
            // This prevents checking this stamp exception for new requests.
            let mut user = user;
            user.reset_stamp_exception();
            if let Err(e) = user.save(&conn).await {
                error!("Error updating user: {e:#?}");
            }
            err_handler!("Stamp exception is expired")
        } else if !stamp_exception.routes.contains(&current_route.to_string()) {
            err_handler!("Invalid security stamp: Current route and exception route do
        } else if stamp_exception.security_stamp != claims.sstamp {
            err_handler!("Invalid security stamp for matched stamp exception")
        }
    } else {
        err_handler!("Invalid security stamp")
    }
}
```

File: `src/db/models/user.rs:96-100`

```
#[derive(Serialize, Deserialize)]
pub struct UserStampException {
    pub routes: Vec<String>,      // Allowed routes during exception
    pub security_stamp: String,    // Old security stamp
    pub expire: i64,               // Unix timestamp expiration
}
```

Risk Analysis

Attack Vectors: 1. **Race Condition:** Attacker attempts to use old JWT token during the 2-minute exception window while simultaneously triggering operations on allowed routes. 2. **Route Confusion:** If route names change or are mismatched, the exception logic may incorrectly allow/deny access. 3. **Time-of-Check to Time-of-Use (TOCTOU):** Between checking `Utc::now().timestamp()` and processing the request, the exception may expire but still be processed. 4. **Exception Persistence Failure:** If `user.save(&conn).await` fails, expired exceptions remain in database, potentially being checked repeatedly. 5. **JSON Deserialization Failure:** If `serde_json::from_str()` fails silently, the exception is ignored without logging, masking potential attacks or misconfigurations.

Impact: - Session hijacking during multi-step operations - Potential for attackers to reuse old tokens within the 2-minute window - Bypass of security stamp validation in edge cases - Audit trail gaps if exception logic fails silently

Likelihood: Low to Medium (requires precise timing and knowledge of internal route structure)

Exploitation Scenario

```
# Hypothetical attack: Exploiting stamp exception window

import requests
import time
from datetime import datetime, timedelta

# 1. Attacker steals user's JWT token (e.g., via XSS, network sniffing)
stolen_jwt = "eyJhbGc..."

# 2. User initiates account recovery, triggering stamp exception
#    (stamp_exception created with 2-minute window)

# 3. Attacker monitors for security stamp change
#    (e.g., via side-channel, timing analysis, or repeated API calls)

# 4. Within the 2-minute window, attacker uses stolen JWT on allowed routes
headers = {"Authorization": f"Bearer {stolen_jwt}"}

# Attempt to access allowed routes during exception window
allowed_routes = [
    "/api/accounts/security-stamp",
    "/api/accounts/kdf",
    "/api/accounts/keys",
]

for route in allowed_routes:
    response = requests.get(f"https://vault.example.com{route}", headers=headers)
    if response.status_code == 200:
        print(f"[!] Access granted to {route} using old token!")
        # Attacker may be able to extract sensitive account data
```

Real-world Risk: This is primarily a concern during account recovery or security key registration flows where users are actively changing credentials.

Remediation

Priority: Medium

Immediate Actions: 1. **Enhanced logging** for stamp exception usage:

```
if Utc::now().timestamp() > stamp_exception.expire {
    warn!("Stamp exception expired for user {} (age: {}s)",
          user.uuid, Utc::now().timestamp() - stamp_exception.expire);
    // ... existing cleanup code
}

// Log successful stamp exception usage
info!("Stamp exception used by user {} on route {} (remaining: {}s)",
      user.uuid, current_route, stamp_exception.expire - Utc::now().timestamp());
```

1. **Reduce exception window:** Consider reducing from 2 minutes to 60-90 seconds for tighter security.
2. **Rate limiting:** Add rate limiting specifically for requests using stamp exceptions to prevent abuse:

```
// Pseudo-code
if stamp_exception_used && !rate_limiter.check_stamp_exception(user_id) {
    err_handler!("Too many requests using stamp exception")
}
```

Long-term Actions: 1. **Simplify exception logic:** Refactor to use single-use tokens instead of time-based exceptions:

```
pub struct StampException {
    pub one_time_token: String,  // Single-use token (consumed on first use)
    pub allowed_route: String,   // Single route, not array
    pub created_at: i64,
}
```

1. **Atomic operations:** Use database transactions to ensure exception cleanup is atomic:

```
conn.transaction(|conn| {
    // Check and consume exception in single transaction
    if let Some(exception) = get_and_delete_exception(user_id, conn)? {
        // Process request
    }
})
```

1. **Explicit exception creation:** Log all stamp exception creations with detailed audit trail:

```
log_event(EventType::UserStampExceptionCreated, user_id, reason="account_recovery");
```

Compensating Controls: - Monitor for repeated stamp exception usage (potential abuse indicator) - Alert on stamp exceptions lasting more than expected duration - Implement additional verification for sensitive operations during exception window

Testing:

```
#[tokio::test]
async fn test_stamp_exception_expiration_race_condition() {
    // Test that expired exceptions are properly rejected even with timing attacks
}

#[tokio::test]
async fn test_stamp_exception_route_validation() {
    // Test that only explicitly allowed routes work during exception
}
```

M-3: Password Hints Stored in Plaintext

Severity: Medium **CWE:** CWE-312 (Cleartext Storage of Sensitive Information) **SOC 2 Relevance:** Yes (CC6.1 - Logical and Physical Access Controls)

Description

Vaultwarden supports an optional password hint feature that allows users to store hints to help remember their master passwords. These hints are stored in plaintext in the database and can be retrieved via email or (when SMTP is disabled) displayed directly in the web interface. This presents an information disclosure risk, as password hints often contain partial passwords, security question answers, or other sensitive information that aids brute-force attacks.

Affected Code

File: `src/db/models/user.rs:42`

```
pub struct User {
    // ...
    pub password_hash: Vec<u8>,
    pub salt: Vec<u8>,
    pub password_iterations: i32,
    pub password_hint: Option<String>,  // ⚠️ Stored in plaintext
    // ...
}
```

Configuration: `.env.template:310`

```
# Controls whether users can set or show password hints.
# This setting applies globally to all users.
# PASSWORD_HINTS_ALLOWED=true  # ⚠️ Enabled by default
```

Password Hint Display: `.env.template:312-316`

```
# Controls whether a password hint should be shown directly in the web page if
# SMTP service is not configured and password hints are allowed.
# Not recommended for publicly-accessible instances because this provides
# unauthenticated access to potentially sensitive data.
# SHOW_PASSWORD_HINT=false
```

Risk Analysis

Attack Vectors: 1. **Database Breach:** If the database is compromised (e.g., SQL injection in another application sharing the database, backup exposure, insider threat), password hints are immediately accessible in plaintext. 2. **Unauthenticated Access:** If `SHOW_PASSWORD_HINT=true` and SMTP is disabled, password hints are displayed on the login page without authentication. 3. **Email Interception:** Password hints sent via email are vulnerable to email server compromise, TLS downgrade attacks, or email forwarding rules set by attackers. 4. **Brute Force Enhancement:** Attackers use hints to generate targeted password lists: - Hint: "pet name + birth year" → Generate wordlist: [Fluffy1985, Spot1990, ...] - Hint: "favorite song" → Use common song titles in attack 5. **Social Engineering:** Hints may reveal personal information useful for phishing or security question attacks.

Common Weak Hints: - "first 4 letters + birth year" (directly describes password structure) - "mom's maiden name + 123" (combines personal info with common pattern) - "Same as email password" (credential reuse indicator) - "The usual one" (suggests reused password across services)

Impact: - Significantly reduces entropy of brute-force attacks - Information disclosure of personal data (names, dates, locations) - Aids offline password cracking if password hash is obtained - Potential SOC 2 compliance issue (insecure storage of authentication data)

Likelihood: Medium (depends on user behavior and configuration)

Exploitation Scenario

```
# Scenario 1: Database breach
# Attacker gains read access to database via:
# - SQL injection in co-hosted application
# - Exposed database backup
# - Compromised database credentials

$ sqlite3 vaultwarden.db
sqlite> SELECT email, password_hint FROM users WHERE password_hint IS NOT NULL;

user1@example.com|pet name + 2020
user2@example.com|first 4 letters of last name + birth year
user3@example.com|favorite band + !
admin@example.com|usual password

# Attacker generates targeted wordlists
$ python3 generate_wordlist.py --hint "pet name + 2020" \
  --pet-names pets.txt --year 2020 > custom_wordlist.txt

# Produces: Fluffy2020, Spot2020, Max2020, Bella2020, ...

# Offline attack with hashcat
$ hashcat -m 10900 -a 0 hash.txt custom_wordlist.txt
```

```
# Scenario 2: Unauthenticated hint disclosure (SHOW_PASSWORD_HINT=true)

import requests

# No authentication required if SHOW_PASSWORD_HINT is enabled
response = requests.post("https://vault.example.com/api/accounts/password-hint",
                         json={"email": "victim@example.com"})

if response.status_code == 200:
    print(f"Password hint: {response.text}")
    # "Password hint: Pet name + favorite number"

    # Use hint to generate targeted password list
    generate_wordlist(hint=response.text,
                      pet_names_file="common_pets.txt",
                      numbers_range=range(0, 100))
```

Real-world Impact Example: - User sets hint: "summer2019!" (too specific, reveals password structure) - Attacker tries: Summer2019!, summer2019!, SUMMER2019! - Password cracked in 3 attempts vs. billions for random password

Remediation

Priority: High

Immediate Actions: 1. **Change default configuration:** Disable password hints by default in future releases:

```
# .env.template
PASSWORD_HINTS_ALLOWED=false  # Changed from true
```

1. **Add security warnings** when hints are enabled:

```
if CONFIG.password_hints_allowed() {
    warn!("⚠️  PASSWORD HINTS ARE ENABLED - This reduces security!");
    warn!("⚠️  Hints are stored in plaintext and aid attackers.");
    warn!("⚠️  Consider disabling: PASSWORD_HINTS_ALLOWED=false");
}

if CONFIG.show_password_hint() {
    error!("⚠️  SHOW_PASSWORD_HINT=true provides unauthenticated access to hints!");
    error!("⚠️  This is NOT RECOMMENDED for public instances.");
}
```

1. **Hint quality enforcement:** If hints remain enabled, enforce minimum quality standards:

```
fn validate_password_hint(hint: &str) -> Result<(), Error> {
    let hint_lower = hint.to_lowercase();

    // Reject hints that are too revealing
    let forbidden_patterns = [
        "first", "last", "letters", "digits", "birth", "year",
        "same as", "usual", "normal", "password", "pass",
    ];

    for pattern in forbidden_patterns {
        if hint_lower.contains(pattern) {
            err!("Password hint is too revealing. Please use a vaguer hint.");
        }
    }

    // Require minimum ambiguity
    if hint.len() < 10 {
        err!("Password hint must be at least 10 characters (to encourage vague hints)")
    }

    Ok(())
}
```

Long-term Actions: 1. **Deprecate password hints entirely:** Modern best practice is to use password managers (which Vaultwarden provides) instead of hints. Consider: - Displaying

deprecation warning in UI - Removing hint feature in major version update - Providing migration guide to secure alternatives

1. **Alternative: Encrypted hints** (if feature must remain):

```
// Encrypt hint with user's master key (client-side)
pub struct User {
    pub password_hint_encrypted: Option<String>,  // Client-encrypted
    pub password_hint_nonce: Option<String>,      // Encryption nonce
}

// Hints become useless without user's master password
// This maintains zero-knowledge architecture
```

1. **Rate limiting:** Implement aggressive rate limiting on hint retrieval:

```
// Allow only 3 hint requests per email per hour
if !rate_limiter.check_hint_request(email, 3, 3600) {
    err!("Too many password hint requests. Try again later.");
}
```

Compensating Controls: - User education: Display warning when users set hints - SMTP requirement: Force `SHOW_PASSWORD_HINT=false` for public instances - Audit logging: Log all password hint retrievals with IP address - Monitoring: Alert on bulk hint retrieval attempts

UI Warning Example:

```
⚠️ Password Hint Security Warning

Password hints are stored unencrypted and may be visible to:
- System administrators
- Attackers who breach the database
- Anyone who intercepts your email

Hints significantly weaken your account security.

Recommended: Leave this field blank and use account recovery instead.

[x] I understand the risks and want to set a hint anyway
```

Testing:

```
#[test]
fn test_password_hint_quality_enforcement() {
    assert!(validate_password_hint("first 4 letters").is_err());
    assert!(validate_password_hint("birth year + pet").is_err());
    assert!(validate_password_hint("Something I remember from childhood").is_ok());
}
```

Low Severity Findings

L-1: Limited Admin Action Audit Logging

Severity: Low **CWE:** CWE-778 (Insufficient Logging) **SOC 2 Relevance:** Yes (CC7.2 - System Monitoring)

Description

While Vaultwarden implements comprehensive event logging for user actions within organizations (35+ event types), admin panel actions have limited audit logging. Critical administrative operations such as user account deletion, 2FA removal, configuration changes, and SSO user management may not be fully logged with sufficient detail for forensic analysis or compliance audits.

Affected Code

File: `src/api/admin.rs` - Various admin endpoints lack event logging

Examples of admin actions with insufficient logging:

```
// User deletion - no event logged
#[post("/users/<user_uuid>/delete")]
async fn delete_user(user_uuid: UserId, _token: AdminToken, conn: DbConn) -> EmptyResul
    User::delete_user(&user_uuid, &conn).await?;
    Ok(())
}

// 2FA removal - no detailed event
#[post("/users/<user_uuid>/remove-2fa")]
async fn remove_2fa(user_uuid: UserId, _token: AdminToken, conn: DbConn) -> EmptyResult
    TwoFactor::delete_all_by_user(&user_uuid, &conn).await?;
    Ok(())
}

// Configuration changes - no event logged
#[post("/config", data = "<data>")]
async fn post_config(data: Json<ConfigBuilder>, _token: AdminToken, conn: DbConn) -> Em
    let data: ConfigBuilder = data.into_inner();
    CONFIG.update_config(data)?;
    Ok(())
}
```

File: `src/api/core/events.rs:41-56` - Org events enabled check

```
let events_json: Vec<Value> = if !CONFIG.org_events_enabled() {
    Vec::with_capacity(0)  // ⚠️ Events disabled by default
} else {
    // Event collection logic
};
```

Configuration: `src/config.rs:629`

```
org_events_enabled: bool, false, def, false;  // ⚠️ Disabled by default
```

Risk Analysis

Impact: - **Compliance Issues:** SOC 2 CC7.2 requires monitoring of system components - **Forensic Gaps:** Unable to determine who performed administrative actions during incident response - **Accountability:** No audit trail for admin abuse or mistakes - **Repudiation:** Admins

can deny performing actions without proof - **Insider Threat Detection:** Cannot detect malicious admin behavior patterns

Missing Log Data: - Admin login/logout events (partially logged) - User account creation/deletion by admin - 2FA bypass/removal by admin - Organization deletion - Configuration changes (security settings, SMTP, SSO, etc.) - Mass operations (bulk user updates) - Database backups performed via admin panel - SMTP test emails sent

Exploitation Scenario

```
# Scenario: Malicious or compromised admin account

# 1. Attacker gains admin panel access (via stolen ADMIN_TOKEN)
# 2. Attacker performs reconnaissance - no logging
curl -X GET https://vault.example.com/admin/users -H "Cookie: VW_ADMIN=$JWT"

# 3. Attacker disables 2FA for target user - insufficient logging
curl -X POST https://vault.example.com/admin/users/$UUID/remove-2fa \
     -H "Cookie: VW_ADMIN=$JWT"

# 4. Attacker modifies configuration to enable signups - no logging
curl -X POST https://vault.example.com/admin/config \
     -H "Cookie: VW_ADMIN=$JWT" \
     -d '{"signups_allowed": true}'

# 5. Attacker creates backdoor account - partial logging only
curl -X POST https://vault.example.com/admin/invite \
     -H "Cookie: VW_ADMIN=$JWT" \
     -d 'attacker@evil.com'

# 6. Attacker cleans up by disabling signups again - no logging
curl -X POST https://vault.example.com/admin/config \
     -H "Cookie: VW_ADMIN=$JWT" \
     -d '{"signups_allowed": false}'

# Result: Minimal forensic evidence of compromise
# Admin audit trail shows only invitation email sent, missing context
```

Remediation

Priority: Medium

Immediate Actions:

1. **Add dedicated admin event types:**

```
// src/db/models/event.rs
pub enum EventType {
    // ... existing user/org events ...

    // Admin panel events (new)
    AdminLogin = 9000,
    AdminLogout = 9001,
    AdminUserDeleted = 9002,
    Admin2FARemoved = 9003,
    AdminConfigChanged = 9004,
    AdminUserDisabled = 9005,
    AdminUserEnabled = 9006,
    AdminOrgDeleted = 9007,
    AdminDatabaseBackup = 9008,
    AdminSMTPTest = 9009,
    AdminMembershipChanged = 9010,
}
```

1. **Implement admin logging helper:**

```
// src/api/admin.rs
async fn log_admin_action(
    action: EventType,
    target_user: Option<&UserId>,
    details: Option<Value>,
    ip: &ClientIp,
    conn: &DbConn,
) -> Result<(), Error> {
    let event = Event::new(
        action,
        None,  // acting_user is admin (use ACTING_ADMIN_USER constant)
        target_user.map(|u| u.clone()),
        details,
        ip.ip.to_string(),
    );
    event.save(conn).await?;
    Ok(())
}
```

1. **Add logging to critical admin endpoints:**

```
#[post("/users/<user_uuid>/delete")]
async fn delete_user(
    user_uuid: UserId,
    _token: AdminToken,
    ip: ClientIp,
    conn: DbConn
) -> EmptyResult {
    let user = User::find_by_uuid(&user_uuid, &conn).await?;

    log_admin_action(
        EventType::AdminUserDeleted,
        Some(&user_uuid),
        Some(json!({"email": user.email})),
        &ip,
        &conn,
    ).await?;

    User::delete_user(&user_uuid, &conn).await?;
    Ok(())
}

#[post("/config", data = "<data>")]
async fn post_config(
    data: Json<ConfigBuilder>,
    _token: AdminToken,
    ip: ClientIp,
    conn: DbConn,
) -> EmptyResult {
    let data: ConfigBuilder = data.into_inner();

    // Log configuration changes with before/after diff
    let old_config = CONFIG.prepare_json();
    CONFIG.update_config(data.clone())?;
    let new_config = CONFIG.prepare_json();

    log_admin_action(
        EventType::AdminConfigChanged,
        None,
        Some(json!({
            "changes": compute_config_diff(&old_config, &new_config)
```

```
        })),
        &ip,
        &conn,
    ).await?;

    Ok(())
}
```

1. **Admin audit log viewer:** Create dedicated admin endpoint for audit trail:

```
#[get("/admin/audit?<start>&<end>")]
async fn get_admin_audit(
    start: String,
    end: String,
    _token: AdminToken,
    conn: DbConn,
) -> JsonResult {
    let start_date = parse_date(&start);
    let end_date = parse_date(&end);

    let events = Event::find_admin_events(&start_date, &end_date, &conn).await;
    Ok(Json(json!({"events": events})))
}
```

Long-term Actions:

1. **Centralized audit logging:** Export admin events to syslog or SIEM:

```
if CONFIG.syslog_enabled() {
    syslog::log_admin_event(event_type, user, ip, details);
}
```

1. **Structured logging with context:**

```
info!(
    admin_action = %event_type,
    target_user = %user_uuid,
    source_ip = %ip,
    session_id = %admin_jwt_id,
    "Admin action performed"
);
```

1. **Compliance-ready log export:**

```
// CSV export for audit reports
#[get("/admin/audit/export?<start>&<end>")]
async fn export_admin_audit(/* ... */) -> Result<NamedFile, Error> {
    // Generate CSV with: timestamp, action, admin_session, target, ip, details
}
```

Compensating Controls: - Enable webserver access logs (captures IP, timestamp, endpoint) - Enable database query logging (captures data changes) - Use systemd journal or syslog for application logs - Implement log aggregation (e.g., ELK stack, Graylog) - Set up alerts for critical admin actions

---

L-2: Missing WebSocket Rate Limiting

Severity: Low **CWE:** CWE-400 (Uncontrolled Resource Consumption) **SOC 2 Relevance:** Partial (CC7.1 - System Operations)

Description

The WebSocket notification service (`/notifications/hub`) requires authentication but does not implement rate limiting on connection attempts or message frequency. While authenticated users are legitimate, this could allow malicious users or compromised accounts to exhaust server resources through WebSocket flooding, connection churn, or message spam.

Affected Code

File: `src/api/notifications.rs:1-100` - WebSocket handler

```
pub fn routes() -> Vec<Route> {
    if CONFIG.enable_websocket() {
        routes![websockets_hub, anonymous_websockets_hub]  // ⚠️ No rate limiting
    } else {
        info!("WebSocket are disabled, realtime sync functionality will not work!");
        routes![]
    }
}

// No rate limiting guard on WebSocket routes
#[get("/hub?<data..>")]
async fn websockets_hub(
    ws: WebSocket,
    data: WsAccessToken,
    ip: ClientIp,
    conn: DbConn,
) -> Result<ws::Stream!['static], Error> {
    // Authentication check present
    let Some(token) = data.access_token else {
        err_code!("Invalid token", 401);
    };

    // But no rate limiting on connections or messages
    // ...
}
```

Comparison: HTTP endpoints have rate limiting:

```
// src/api/identity.rs - Login has rate limiting
#[post("/connect/token", data = "<data>")]
async fn login(
    data: Form<ConnectData>,
    client_header: ClientHeaders,
    conn: DbConn,
) -> JsonResult {
    _check_is_some(&data.client_id, "client_id")?;

    // Rate limiting check
    crate::ratelimit::check_limit_login(&client_header.ip)?;  // ✅ Rate limited
    // ...
}
```

Risk Analysis

Attack Vectors: 1. **Connection Flood:** Attacker opens thousands of WebSocket connections from authenticated session 2. **Connection Churn:** Rapidly connect/disconnect to exhaust connection pool or CPU 3. **Message Spam:** Send high-frequency ping messages to consume bandwidth 4. **Resource Exhaustion:** Each WebSocket consumes memory; large-scale attack exhausts RAM 5. **Amplification:** Compromise multiple user accounts to multiply attack effect

Impact: - Service degradation or denial of service - Increased infrastructure costs (bandwidth, CPU, memory) - Legitimate users unable to connect or receive real-time notifications - Potential crash or restart of vaultwarden service

Likelihood: Low (requires authenticated users, limited attack benefit)

Exploitation Scenario

```
import asyncio
import websockets
import json

# Attacker has valid access token (authenticated user or compromised account)
ACCESS_TOKEN = "eyJhbGc..."

async def websocket_flood():
    """Open many WebSocket connections to exhaust resources"""
    connections = []

    for i in range(1000):  # Open 1000 connections
        try:
            uri = f"wss://vault.example.com/notifications/hub?access_token={ACCESS_TOKE
            ws = await websockets.connect(uri)
            connections.append(ws)
            print(f"Connection {i} established")
        except Exception as e:
            print(f"Failed: {e}")
            break

    # Keep connections alive
    await asyncio.sleep(3600)

# Each connection consumes ~5-10KB memory + file descriptor
# 1000 connections = ~10MB + 1000 FDs
# Scaled across multiple accounts = significant resource exhaustion

asyncio.run(websocket_flood())
```

```
# Connection churn attack (rapid connect/disconnect)
while true; do
  websocat "wss://vault.example.com/notifications/hub?access_token=$TOKEN" &
  sleep 0.1
  kill $!
done

# Result: CPU exhaustion from TLS handshakes and connection setup/teardown
```

Remediation

Priority: Low to Medium

Immediate Actions:

1. **Connection-level rate limiting:**

```
use rocket::request::{FromRequest, Outcome, Request};
use std::sync::Arc;
use governor::{Quota, RateLimiter};

// Limit WebSocket connections per user
static WS_RATE_LIMITER: LazyLock<Arc<RateLimiter<UserId>>> = LazyLock::new(|| {
    // Allow 5 connections per user per minute
    Arc::new(RateLimiter::keyed(Quota::per_minute(5)))
});

#[get("/hub?<data..>")]
async fn websockets_hub(
    ws: WebSocket,
    data: WsAccessToken,
    ip: ClientIp,
    conn: DbConn,
) -> Result<ws::Stream!['static], Error> {
    let token = data.access_token.ok_or("Invalid token")?;
    let user_uuid = decode_token(&token)?;

    // Rate limit check
    if !WS_RATE_LIMITER.check_key(&user_uuid).is_ok() {
        err_code!("Too many WebSocket connection attempts. Please wait.", 429);
    }

    // Continue with connection setup
    // ...
}
```

1. **Per-user connection limit:**

```
// Limit concurrent connections per user
const MAX_CONNECTIONS_PER_USER: usize = 10;

async fn websockets_hub(/* ... */) -> Result<ws::Stream!['static], Error> {
    // Check current connection count
    if let Some(entry) = WS_USERS.map.get(user_uuid.as_ref()) {
        if entry.len() >= MAX_CONNECTIONS_PER_USER {
            err_code!("Maximum concurrent WebSocket connections reached", 429);
        }
    }

    // Continue with connection
}
```

1. **Message rate limiting:**

```
// Limit message frequency per connection
async fn handle_websocket_message(
    msg: Message,
    rate_limiter: &RateLimiter,
) -> Result<(), Error> {
    if !rate_limiter.check().is_ok() {
        // Close connection on abuse
        return Err(Error::new("Message rate limit exceeded"));
    }

    // Process message
}
```

Long-term Actions:

1. **Global WebSocket limits:**

```
// Configuration options
org_events_enabled: bool, false, def, false;
websocket_max_connections_per_user: usize, true, def, 10;
websocket_max_global_connections: usize, true, def, 10000;
websocket_rate_limit_per_minute: usize, true, def, 5;
```

1. **Monitoring and alerting:**

```
// Expose WebSocket metrics
#[get("/admin/metrics/websockets")]
fn websocket_metrics(_token: AdminToken) -> JsonResult {
    Ok(Json(json!({
        "total_connections": WS_USERS.map.len(),
        "connections_by_user": get_connection_distribution(),
        "total_messages_per_second": get_message_rate(),
    })))
}
```

1. **Graceful degradation:**

```
// Reject new connections if approaching resource limits
if get_total_websocket_connections() > CONFIG.websocket_max_global_connections() * 0.9
    warn!("WebSocket connection limit approaching, rejecting new connections");
    err_code!("WebSocket service at capacity, please try again later", 503);
}
```

Compensating Controls: - Reverse proxy rate limiting (nginx, Caddy) - Connection timeout enforcement - Firewall-level connection tracking - Monitoring for abnormal connection patterns - Cloudflare or similar DDoS protection

---

L-3: No MFA Option for Admin Panel

Severity: Low **CWE:** CWE-306 (Missing Authentication for Critical Function) **SOC 2 Relevance:** Yes (CC6.1 - Logical and Physical Access Controls)

Description

The admin panel authentication (`/admin`) relies solely on the `ADMIN_TOKEN` without support for multi-factor authentication. While the token can be Argon2id hashed, there is no second factor protection. If the admin token is compromised (e.g., configuration file exposure, keylogger, shoulder surfing), an attacker gains immediate full administrative access without additional verification.

Affected Code

File: `src/api/admin.rs:200-227` - Admin login

```
#[post("/", data = "<data>")]
fn post_admin_login(
    data: Form<LoginForm>,
    cookies: &CookieJar<'_>,
    ip: ClientIp,
    secure: Secure,
) -> Result<Redirect, AdminResponse> {
    // Single-factor authentication only
    if !_validate_token(&data.token) {
        error!("Invalid admin token. IP: {}", ip.ip);
        Err(AdminResponse::Unauthorized(/* ... */))
    } else {
        // Generate JWT and set cookie - no 2FA challenge
        let claims = generate_admin_claims();
        let jwt = encode_jwt(&claims);
        cookies.add(cookie);
        // ...
    }
}
```

No 2FA enforcement: - No TOTP verification - No WebAuthn/FIDO2 support for admin - No email-based verification - No IP allowlisting

Risk Analysis

Impact: - Full administrative access if token compromised - Ability to view all user data, delete accounts, modify configuration - Potential for complete system compromise

Likelihood: Low (requires token compromise, which should be rare with proper security)

Attack Scenarios: 1. **Config File Exposure:** `.env` file leaked via misconfigured web server → admin token stolen 2. **Insider Threat:** System administrator with file access uses token maliciously 3. **Shoulder Surfing:** Token observed during configuration or troubleshooting 4. **Backup Exposure:** Old backups containing plain-text admin token accessed

Remediation

Priority: Low to Medium

Option 1: TOTP-based Admin 2FA

```
// Add admin 2FA settings to config
admin_2fa_enabled: bool, true, def, false;
admin_2fa_secret: Option<String>, true, option;

#[post("/", data = "<data>")]
fn post_admin_login(
    data: Form<LoginForm>,
    cookies: &CookieJar<'_>,
    ip: ClientIp,
) -> Result<Redirect, AdminResponse> {
    // Step 1: Validate admin token
    if !_validate_token(&data.token) {
        error!("Invalid admin token. IP: {}", ip.ip);
        return Err(AdminResponse::Unauthorized(/* ... */));
    }

    // Step 2: Check if 2FA is enabled
    if CONFIG.admin_2fa_enabled() {
        if let Some(totp_code) = &data.totp_code {
            if !verify_admin_totp(totp_code) {
                error!("Invalid admin 2FA code. IP: {}", ip.ip);
                return Err(AdminResponse::Unauthorized(/* ... */));
            }
        } else {
            // Redirect to 2FA input page
            return Err(AdminResponse::Require2FA(/* ... */));
        }
    }

    // Generate JWT
    let jwt = encode_jwt(&generate_admin_claims());
    cookies.add(cookie);
    Ok(Redirect::to(admin_path()))
}
```

Option 2: IP Allowlisting

```
// Add IP allowlist to config
admin_ip_allowlist: Vec<String>, true, def, vec![];

fn check_admin_ip_allowed(ip: &IpAddr) -> bool {
    let allowlist = CONFIG.admin_ip_allowlist();

    if allowlist.is_empty() {
        return true;  // No restriction if not configured
    }

    allowlist.iter().any(|allowed| {
        // Support CIDR notation: 192.168.1.0/24
        ip_matches_cidr(ip, allowed)
    })
}

#[post("/", data = "<data>")]
fn post_admin_login(/* ... */, ip: ClientIp) -> Result<Redirect, AdminResponse> {
    // Check IP allowlist first
    if !check_admin_ip_allowed(&ip.ip) {
        error!("Admin access denied from IP: {}", ip.ip);
        return Err(AdminResponse::Unauthorized(/* ... */));
    }

    // Continue with token validation
    // ...
}
```

Configuration:

```
# .env.template
## Admin panel multi-factor authentication
# ADMIN_2FA_ENABLED=false
# ADMIN_2FA_SECRET=base32_secret_here

## Admin panel IP allowlist (comma-separated CIDR blocks)
## Leave empty to allow all IPs
# ADMIN_IP_ALLOWLIST=192.168.1.0/24,10.0.0.0/8,127.0.0.1/32
```

Compensating Controls: - Use strong, randomly generated admin tokens (64+ characters) - Rotate admin token regularly - Use Argon2id hashed tokens (never plain-text) - Enable admin access logging - Use reverse proxy authentication (e.g., Authelia, oauth2-proxy) - Restrict admin panel to VPN-only access - Implement IP-based firewall rules

---

L-4: Event Log Export Functionality Gaps

Severity: Low **CWE:** CWE-778 (Insufficient Logging) **SOC 2 Relevance:** Yes (CC7.2 - System Monitoring)

Description

Vaultwarden's event logging system captures 35+ event types for organizations, but lacks built-in functionality to export these logs in formats suitable for compliance reporting, external SIEM integration, or long-term archival. Event logs are stored in the database and can only be viewed through the web UI with limited filtering and no bulk export capability.

Affected Code

File: `src/api/core/events.rs` - Event retrieval endpoints

```
// Only JSON API access, no CSV/export functionality
#[get("/organizations/<org_id>/events?<data..>")]
async fn get_org_events(
    org_id: OrganizationId,
    data: EventRange,
    headers: AdminHeaders,
    conn: DbConn,
) -> JsonResult {
    // Returns JSON only - no export format options
    let events_json: Vec<Value> = /* ... */;

    Ok(Json(json!({
        "data": events_json,
        "object": "list",
        "continuationToken": get_continuation_token(&events_json),
    })))
}
```

Missing Features: - CSV export for spreadsheet analysis - JSON Lines format for log aggregation tools - Syslog/SIEM integration (syslog library present but not used for events) - Automated log retention and archival - Bulk export API - Filtered exports by event type

Risk Analysis

Impact: - Compliance reporting difficulty (SOC 2 audits require log evidence) - Manual effort to extract and format event data - No automated log archival (potential for data loss) - Limited forensic analysis capabilities - Cannot easily integrate with external monitoring tools

Likelihood: N/A (functionality gap, not a vulnerability)

Remediation

Priority: Low

1. Add CSV export endpoint:

```
use csv::Writer;

#[get("/organizations/<org_id>/events/export?<data..>")]
async fn export_org_events(
    org_id: OrganizationId,
    data: EventRange,
    headers: AdminHeaders,
    conn: DbConn,
) -> Result<Vec<u8>, Error> {
    if org_id != headers.org_id {
        err!("Organization not found");
    }

    let start_date = parse_date(&data.start);
    let end_date = parse_date(&data.end);

    let events = Event::find_by_organization_uuid(&org_id, &start_date, &end_date, &con

    // Generate CSV
    let mut wtr = Writer::from_writer(vec![]);
    wtr.write_record(&["Timestamp", "Event Type", "User", "IP Address", "Device", "Deta

    for event in events {
        wtr.write_record(&[
            event.event_date.to_string(),
            event.event_type.to_string(),
            event.user_uuid.map(|u| u.to_string()).unwrap_or_default(),
            event.ip_address,
            event.device_type,
            event.details.unwrap_or_default(),
        ])?;
    }

    Ok(wtr.into_inner()?)
}
```

2. Syslog integration:

```
use syslog::{Facility, Formatter3164};

fn send_event_to_syslog(event: &Event) {
    if !CONFIG.syslog_enabled() {
        return;
    }

    let formatter = Formatter3164 {
        facility: Facility::LOG_USER,
        hostname: None,
        process: "vaultwarden".into(),
        pid: 0,
    };

    match syslog::unix(formatter) {
        Ok(mut writer) => {
            let _ = writer.info(format!(
                "event_type={} user={} org={} ip={} details={}",
                event.event_type,
                event.user_uuid.unwrap_or_default(),
                event.organization_uuid.unwrap_or_default(),
                event.ip_address,
                event.details.unwrap_or_default(),
            ));
        }
        Err(e) => error!("Failed to send syslog: {}", e),
    }
}
```

3. Automated log archival:

```
// Background job to archive old events
async fn archive_old_events(conn: &DbConn) -> Result<(), Error> {
    let retention_days = CONFIG.event_retention_days();
    let cutoff_date = Utc::now() - TimeDelta::days(retention_days);

    let old_events = Event::find_before_date(&cutoff_date, conn).await?;

    // Export to file before deletion
    let archive_path = format!("{}/events_archive_{}.jsonl",
                               CONFIG.data_folder(),
                               Utc::now().format("%Y%m%d"));

    let file = File::create(archive_path)?;
    for event in &old_events {
        serde_json::to_writer(&file, &event.to_json())?;
        writeln!(&file)?;
    }

    // Delete from database
    Event::delete_before_date(&cutoff_date, conn).await?;

    info!("Archived {} old events", old_events.len());
    Ok(())
}
```

Compensating Controls: - Manual database exports: `sqlite3 vaultwarden.db ".mode csv" "SELECT * FROM events"` - API scripting to fetch events via JSON and convert externally - Database replication to separate log database - Regular database backups

---

L-5: Input Validation Edge Cases

Severity: Low **CWE:** CWE-20 (Improper Input Validation) **SOC 2 Relevance:** Partial (CC6.1 - Logical and Physical Access Controls)

Description

While Vaultwarden employs strong input validation in most areas (Diesel ORM, JSON schema validation, type safety via Rust), there are minor edge cases where input validation could be

more restrictive to prevent abuse or unexpected behavior. These include email address format validation, organization name length limits, and collection name constraints.

Affected Areas

1. Email Address Validation

Currently accepts extremely long or malformed emails that pass basic regex but may cause issues:

```
// Overly permissive email validation may accept:
// - Very long emails (>254 characters per RFC, but no enforcement)
// - Multiple @ symbols in local part (technically valid but unusual)
// - Unicode characters without normalization
```

2. Organization/Collection Names

No apparent length limits on organization or collection names:

```
// Could allow:
organization.name = "A" * 10000  // 10KB organization name
collection.name = "Very long collection name..." * 1000
```

3. User Display Names

Similar issues with user name fields allowing potentially abusive lengths or characters.

Risk Analysis

Impact: - UI rendering issues with extremely long names - Database performance degradation - Potential for stored XSS if names contain unusual Unicode - Log file pollution with excessively long strings

Likelihood: Very Low (requires authenticated user, limited impact)

Remediation

Priority: Low

1. Email validation enhancement:

```
fn validate_email(email: &str) -> Result<(), Error> {
    // Length check (RFC 5321: 254 characters max)
    if email.len() > 254 {
        err!("Email address too long (max 254 characters)");
    }

    // Basic format check
    let email_regex = regex::Regex::new(r"^[^\s@]+@[^\s@]+\.[^\s@]+$")?;
    if !email_regex.is_match(email) {
        err!("Invalid email format");
    }

    // Normalize Unicode
    let normalized = email.nfc().collect::<String>();

    // Check for suspicious patterns
    if email.matches('@').count() > 1 {
        warn!("Email with multiple @ symbols: {}", email);
    }

    Ok(())
}
```

2. Name length limits:

```
const MAX_ORG_NAME_LENGTH: usize = 100;
const MAX_COLLECTION_NAME_LENGTH: usize = 100;
const MAX_USER_NAME_LENGTH: usize = 100;

fn validate_organization_name(name: &str) -> Result<(), Error> {
    if name.is_empty() {
        err!("Organization name cannot be empty");
    }

    if name.len() > MAX_ORG_NAME_LENGTH {
        err!("Organization name too long (max {} characters)", MAX_ORG_NAME_LENGTH);
    }

    // Check for control characters
    if name.chars().any(|c| c.is_control()) {
        err!("Organization name contains invalid characters");
    }

    Ok(())
}
```

3. Input sanitization:

```
fn sanitize_display_text(input: &str) -> String {
    input
        .chars()
        .filter(|c| !c.is_control() || *c == '\n')  // Allow newlines only
        .take(1000)  // Hard limit
        .collect()
}
```

Compensating Controls: - Frontend validation (already present in web vault) - Database column length constraints - UI truncation of long names - Monitoring for abnormally long inputs

---

Summary and Recommendations

Findings Summary

Severity	Count	Key Issues
Medium	3	Plain-text admin tokens, security stamp complexity, password hints
Low	5	Audit logging gaps, WebSocket rate limiting, admin MFA, log export, input validation

Priority Remediation Roadmap

Immediate Actions (1-2 weeks): 1. Add startup warnings for plain-text ADMIN_TOKEN (**M-1**) 2. Disable PASSWORD_HINTS_ALLOWED by default in next release (**M-3**) 3. Enhance admin action audit logging (**L-1**) 4. Add WebSocket connection rate limiting (**L-2**)

Short-term Actions (1-3 months): 1. Provide admin token hashing utility (**M-1**) 2. Simplify security stamp exception logic (**M-2**) 3. Implement CSV event log export (**L-4**) 4. Add admin panel IP allowlisting (**L-3**)

Long-term Actions (6-12 months): 1. Deprecate plain-text admin tokens entirely (breaking change) (**M-1**) 2. Consider deprecating password hints feature (**M-3**) 3. Implement admin panel MFA option (**L-3**) 4. Enhance input validation framework (**L-5**)

SOC 2 Compliance Considerations

Areas Requiring Attention: - **CC6.1 (Access Controls):** M-1, M-3, L-3 all impact access control security - **CC7.2 (System Monitoring):** L-1 and L-4 affect audit logging and monitoring capabilities

Recommendations for SOC 2 Readiness: 1. Enforce Argon2id admin tokens (eliminate plain-text acceptance) 2. Disable password hints by default or encrypt them client-side 3. Implement comprehensive admin action logging 4. Provide audit log export functionality (CSV/JSON) 5. Add MFA option for admin panel access 6. Document security configurations in system security policies

Testing Recommendations

Security Testing: 1. Penetration test admin panel authentication (brute force, token exposure scenarios) 2. Stress test WebSocket service (connection flooding, message spam) 3. Verify

security stamp exception logic under race conditions 4. Test input validation with fuzzing tools (AFL, libFuzzer)

Compliance Testing: 1. Audit trail completeness review 2. Log retention and export testing 3. Access control policy enforcement verification 4. Configuration security review

---

Conclusion

Vaultwarden's Medium and Low severity findings represent opportunities for security hardening rather than critical vulnerabilities. The application maintains a strong security posture overall, with these findings primarily addressing: - **Legacy compatibility trade-offs** (plain-text admin tokens) - **Complexity management** (security stamp exceptions) - **Defense-in-depth enhancements** (rate limiting, MFA, logging) - **Compliance readiness** (audit logging, log export)

Implementing the recommended remediations will significantly strengthen Vaultwarden's security posture and align it with enterprise security best practices and SOC 2 compliance requirements.

End of Report 3: Medium & Low Severity Findings

Next Report: Informational Findings & Remediation Recommendations (complete security hardening guide)